

Lab Assignment 10

Lab: Applications of Edmonds–Karp Algorithm

Reference: For the implementation and theoretical background of the Edmonds–Karp algorithm, refer to [Edmonds–Karp Algorithm \(CP-Algorithms\)](#).

The Edmonds–Karp algorithm is an implementation of the Ford–Fulkerson method for computing the maximum flow in a flow network. It uses Breadth-First Search (BFS) to find augmenting paths and guarantees a polynomial running time of $O(VE^2)$. In this lab, you will implement the Edmonds–Karp algorithm and apply it to different graph problems.

Problem 1: Implementing Edmonds–Karp and Finding Minimum Cut

Objective: Implement the Edmonds–Karp algorithm to find the maximum flow in a directed graph, identify the corresponding minimum $s-t$ cut, and verify whether the minimum cut is unique.

Description: Given a directed graph $G = (V, E)$ with source s and sink t , each edge $(u, v) \in E$ has a positive capacity $c(u, v)$. Your tasks:

1. Implement the Edmonds–Karp algorithm to compute the maximum flow from s to t .
2. Identify the edges that form a minimum-capacity $s-t$ cut.
3. Determine whether the minimum cut is unique. For a helpful discussion see [Stack-Exchange discussion on deciding uniqueness of a min-cut](#) “Decide whether a flow graph has a single min-cut” (CS.StackExchange).

Hint: After computing the max-flow, perform a BFS/DFS from the source in the residual graph. The reachable set defines one side of the minimum cut. Use the linked resource above to reason about uniqueness.

Expected Output:

- Maximum flow value.
- Set of edges forming the minimum cut.
- Statement on whether the minimum cut is unique.

Problem 2: Maximum Matching Using Flow

Objective: Use the Edmonds–Karp algorithm to find a maximum matching in a bipartite graph.

Description: Given a bipartite graph $G = (U, V, E)$, where each edge connects a vertex in U to one in V . Find the maximum matching — the largest set of edges no two of which share an endpoint.

Flow Network Construction:

1. Create source s , connect $s \rightarrow u$ for all $u \in U$ with capacity 1.
2. Create sink t , connect $v \rightarrow t$ for all $v \in V$ with capacity 1.
3. For each $(u, v) \in E$, add directed edge $u \rightarrow v$ with capacity 1.

Steps:

1. Run Edmonds–Karp on this network to obtain maximum flow.
2. Each unit of flow from U to V corresponds to one matched pair.
3. Output the matching pairs.

Expected Output:

- Maximum matching size.
- List of matched pairs (u, v) .

Example: If $U = \{u_1, u_2, u_3\}$, $V = \{v_1, v_2, v_3\}$ and edges $\{(u_1, v_1), (u_1, v_2), (u_2, v_2), (u_3, v_3)\}$, a possible output is matching size 3: $\{(u_1, v_1), (u_2, v_2), (u_3, v_3)\}$.

Problem 3: Disjoint Paths Using Flow

Objective: Apply the Edmonds–Karp algorithm to find maximum number of disjoint paths between two vertices s and t .

Part (a): Vertex-disjoint Paths in a Directed Graph

Description: Given directed graph $G = (V, E)$ and vertices s, t . Find the maximum number of vertex-disjoint $s \rightarrow t$ paths.

Hint: Use the vertex-splitting transformation: for each $v \notin \{s, t\}$ replace by v_1 and v_2 ; add edge $v_1 \rightarrow v_2$ of capacity 1; redirect all incoming edges of v to v_1 , all outgoing edges from v out of v_2 ; set the redirected edges' capacities to a high value (or $|V|$). Running the flow algorithm gives the number of vertex-disjoint paths.

Part (b): Vertex-disjoint Paths in an Undirected Graph Given undirected graph $G = (V, E)$ and vertices s, t . Convert each undirected edge (u, v) into two directed edges $u \rightarrow v$ and $v \rightarrow u$ (each of capacity 1). Then apply the vertex-splitting transformation as above and run the flow algorithm.

Part (c): Edge-disjoint Paths You may optionally extend the above to compute maximum number of **edge-disjoint** paths. For reference and further reading, see <https://www.cs.cmu.edu/~ckingsf/bioinfo-lectures/edgedisjoint.pdf> Edge-Disjoint Paths lecture (CMU) .

Expected Output:

- Number of disjoint paths between s and t .
- Optionally, list each such path as reconstructed from the residual flow.

Note: These three problems illustrate how the same core algorithm — the Edmonds–Karp method — can be used for very different graph-theoretic tasks: minimum cut, bipartite matching, and path disjointness.